

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 10
“FIND MAX/MIN”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement pencarian

1. Latihan1

Source Code:

```
package main

import "fmt"

const NMAX = 1000

func main() {
    var n int
    var berat [NMAX]float64

    fmt.Scan(&n)

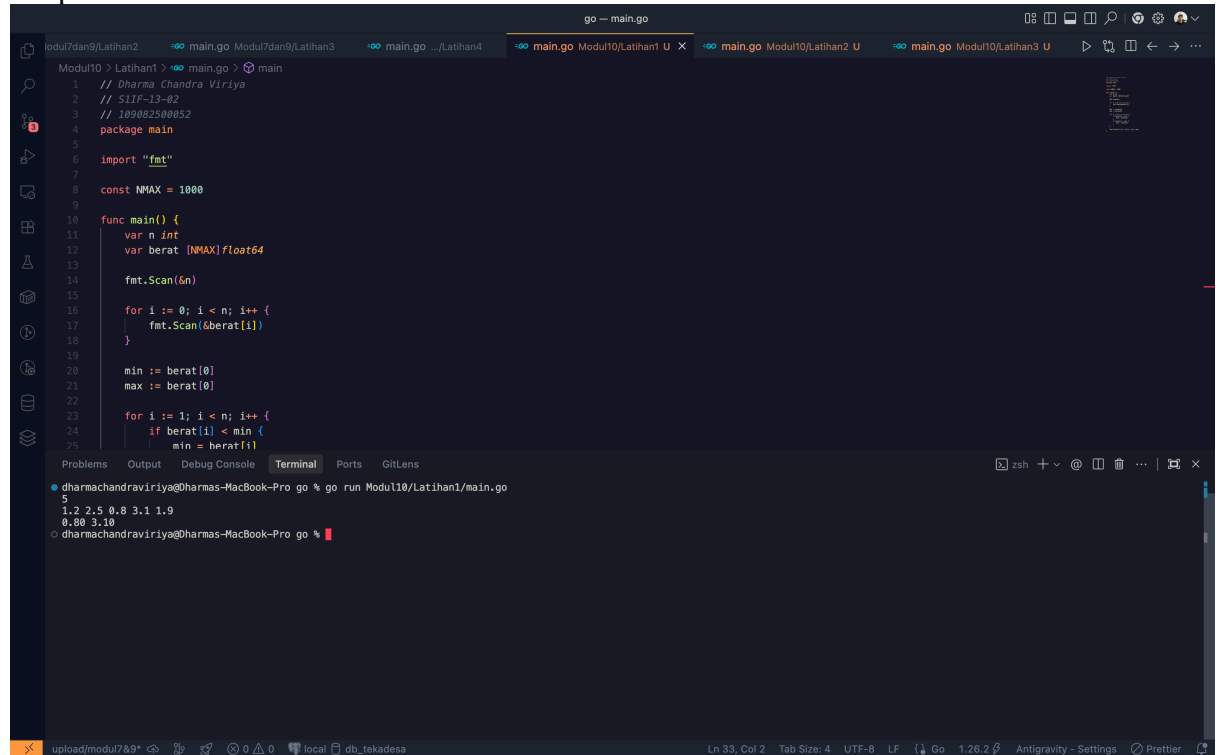
    for i := 0; i < n; i++ {
        fmt.Scan(&berat[i])
    }

    min := berat[0]
    max := berat[0]

    for i := 1; i < n; i++ {
        if berat[i] < min {
            min = berat[i]
        }
        if berat[i] > max {
            max = berat[i]
        }
    }

    fmt.Printf("%.2f %.2f\n", min, max)
}
```

Output:



```
1 // Dharma Chandra Viriya
2 // SIIP-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 const NMAX = 1000
9
10 func main() {
11     var n int
12     var berat [NMAX]float64
13
14     fmt.Scan(&n)
15
16     for i := 0; i < n; i++ {
17         fmt.Scan(&berat[i])
18     }
19
20     min := berat[0]
21     max := berat[0]
22
23     for i := 1; i < n; i++ {
24         if berat[i] < min {
25             min = berat[i]
26         }
27     }
28
29     fmt.Printf("Minimum: %.2f\n", min)
30     fmt.Printf("Maximum: %.2f\n", max)
31 }
```

Terminal Output:

```
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul10/Latihan1/main.go
5
1.2 2.5 0.8 3.1 1.9
0.80 3.10
```

Deskripsi Program:

Program ini dibuat untuk menentukan berat kelinci terkecil dan terbesar dari sekumpulan data berat kelinci yang dimasukkan oleh pengguna. Program menggunakan sebuah array bertipe `float64` dengan kapasitas maksimum 1000 elemen untuk menyimpan data berat kelinci. Pada awal eksekusi, pengguna memasukkan banyaknya data yang akan diproses, kemudian memasukkan berat masing-masing kelinci satu per satu. Setelah seluruh data tersimpan dalam array, program melakukan pencarian nilai minimum dan maksimum dari data tersebut, kemudian menampilkannya sebagai hasil akhir. Secara teknis, program mendeklarasikan konstanta `NMAX` sebagai kapasitas maksimum array dan menggunakan variabel `n` untuk menyimpan jumlah data yang diinput. Data berat kelinci dibaca menggunakan perulangan dan disimpan ke dalam array `berat`. Selanjutnya, elemen pertama array digunakan sebagai nilai awal untuk variabel `min` dan `max`. Program kemudian menelusuri seluruh data mulai dari elemen kedua hingga elemen terakhir. Jika ditemukan nilai yang lebih kecil dari `min`, maka nilai minimum diperbarui. Sebaliknya, jika ditemukan nilai yang lebih besar dari `max`, maka nilai maksimum diperbarui. Metode ini memungkinkan pencarian nilai terkecil dan terbesar dilakukan hanya dalam satu kali penelusuran data sehingga prosesnya efisien. Setelah proses pencarian selesai, nilai minimum dan maksimum ditampilkan dengan format dua angka di belakang koma menggunakan fungsi `fmt.Printf`.

2. Latihan2

Source Code:

```
package main

import "fmt"

const NMAX = 1000

func main() {
    var x, y int
    var ikan [NMAX]float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&ikan[i])
    }

    jumlahWadah := (x + y - 1) / y
    var totalSemua float64

    for w := 0; w < jumlahWadah; w++ {
        var totalWadah float64

        awal := w * y
        akhir := awal + y
        if akhir > x {
            akhir = x
        }

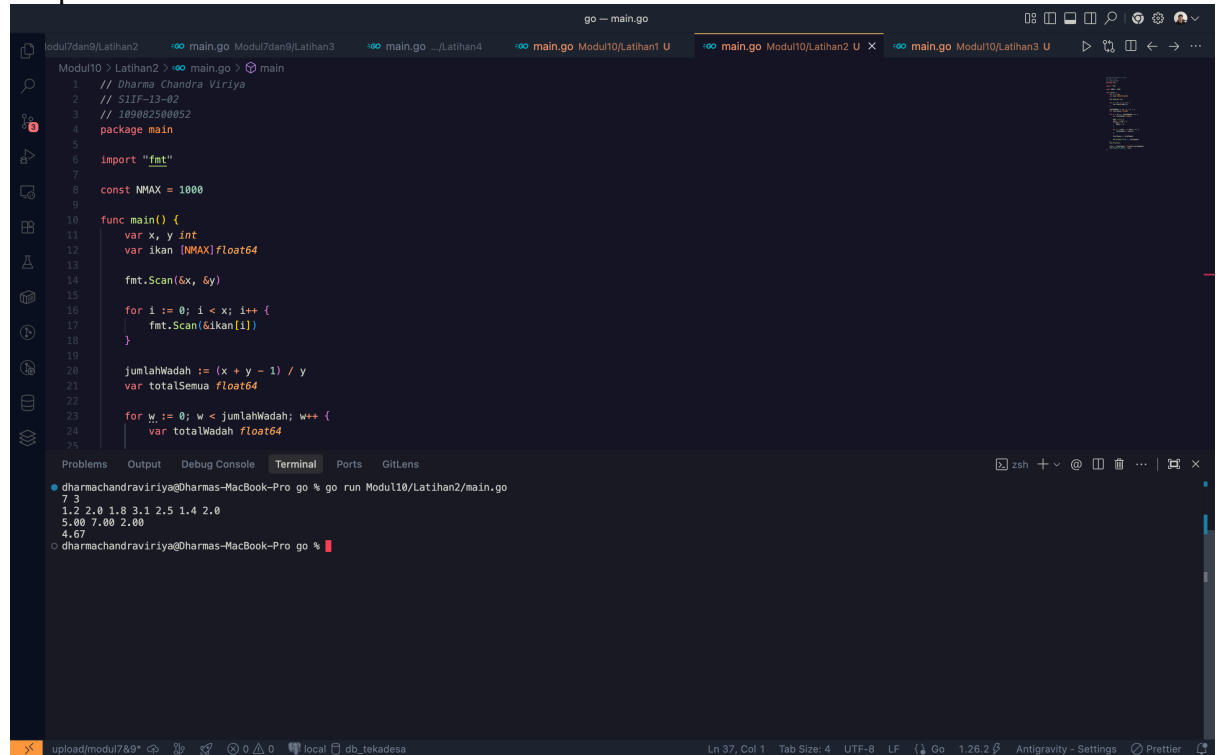
        for i := awal; i < akhir; i++ {
            totalWadah += ikan[i]
        }

        totalSemua += totalWadah

        fmt.Printf("%.2f ", totalWadah)
    }
    fmt.Println()

    rata := totalSemua / float64(jumlahWadah)
    fmt.Printf("%.2f\n", rata)
}
```

Output:



```
Modul10 > Latihan2 > go run Modul10/Latihan2/main.go
1 // Dharma Chandra Viriya
2 // SIIP-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 const NMAX = 1000
9
10 func main() {
11     var x, y int
12     var ikan [NMAX]float64
13
14     fmt.Scan(&x, &y)
15
16     for i := 0; i < x; i++ {
17         fmt.Scan(&ikan[i])
18     }
19
20     jumlahWadah := (x + y - 1) / y
21     var totalSemua float64
22
23     for w := 0; w < jumlahWadah; w++ {
24         var totalWadah float64
25     }
26 }
27
28 Problems Output Debug Console Terminal Ports GitLens
29
30 dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul10/Latihan2/main.go
31 7 3
32 1 2 2.0 1.8 3.1 2.5 1.4 2.0
33 5.00 7.00 2.00
34 4.67
35
36 dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini dibuat untuk menghitung total berat ikan pada setiap wadah serta menentukan rata-rata berat ikan per wadah. Program menggunakan array bertipe `float64` dengan kapasitas maksimum 1000 elemen untuk menyimpan data berat ikan. Pengguna memasukkan dua buah bilangan bulat, yaitu `x` yang menyatakan jumlah ikan dan `y` yang menyatakan kapasitas ikan dalam setiap wadah. Selanjutnya, pengguna memasukkan berat masing-masing ikan yang akan disimpan ke dalam array. Program kemudian mengelompokkan ikan ke dalam beberapa wadah sesuai urutan input dan menghitung total berat ikan pada setiap wadah. Setelah seluruh wadah diproses, program menghitung dan menampilkan rata-rata berat ikan per wadah. Secara teknis, program terlebih dahulu membaca nilai `x` dan `y`, lalu menyimpan data berat ikan ke dalam array `ikan`. Jumlah wadah yang diperlukan dihitung menggunakan rumus $(x + y - 1) / y$, yang menghasilkan pembagian dengan pembulatan ke atas sehingga seluruh ikan dapat ditampung meskipun jumlah ikan tidak habis dibagi kapasitas wadah. Program kemudian melakukan perulangan untuk setiap wadah. Pada setiap iterasi, ditentukan indeks awal dan indeks akhir ikan yang termasuk ke dalam wadah tersebut. Jika indeks akhir melebihi jumlah ikan yang tersedia, nilainya disesuaikan agar tidak keluar dari batas array. Selanjutnya, berat seluruh ikan dalam wadah dijumlahkan dan disimpan pada variabel `totalWadah`. Nilai tersebut ditampilkan sebagai total berat wadah sekaligus ditambahkan ke variabel `totalSemua` untuk keperluan perhitungan rata-rata. Setelah seluruh wadah selesai diproses, rata-rata berat per wadah dihitung dengan membagi `totalSemua` dengan `jumlahWadah`, kemudian hasilnya ditampilkan dengan format dua angka di belakang koma menggunakan fungsi `fmt.Printf`.

3. Latihan3

Source Code:

```
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64

    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }

    return total / float64(n)
}

func main() {
    var data arrBalita
    var n int
    var bMin, bMax float64

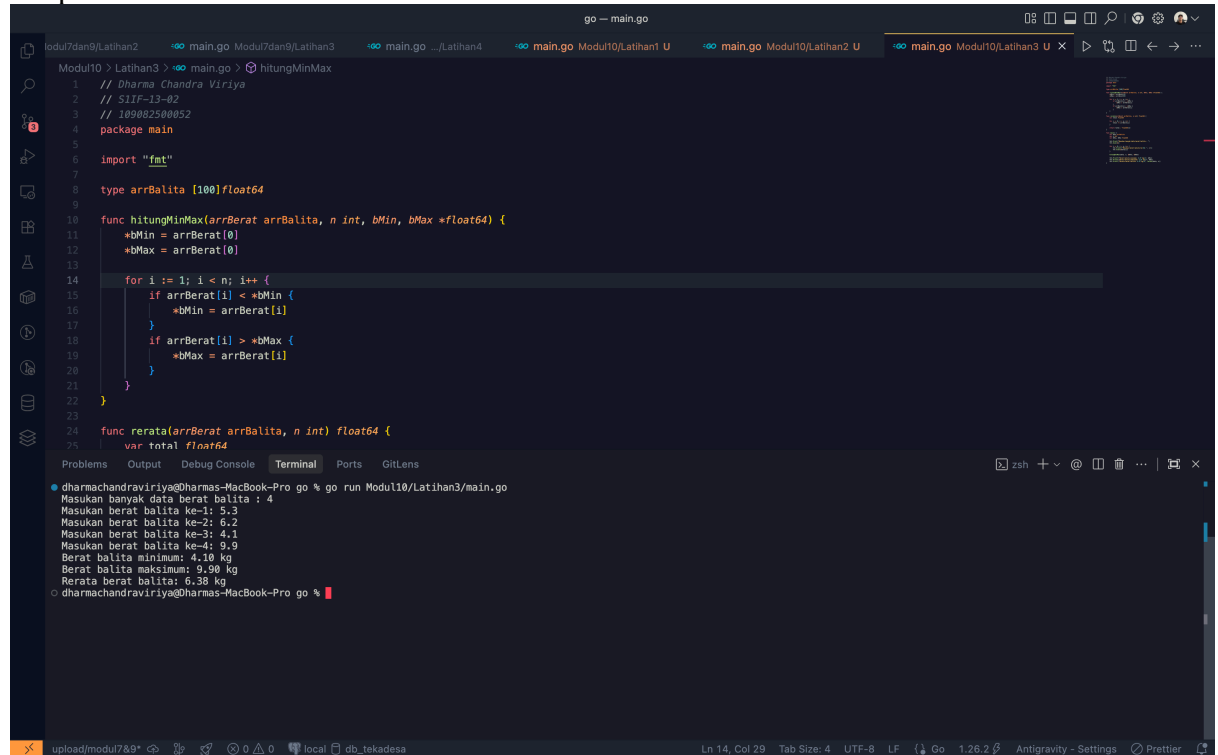
    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    hitungMinMax(data, n, &bMin, &bMax)

    fmt.Printf("Berat balita minimum: %.2f kg\n", bMin)
    fmt.Printf("Berat balita maksimum: %.2f kg\n", bMax)
    fmt.Printf("Rerata berat balita: %.2f kg\n", rerata(data, n))
}
```

Output:



```
Modul10 > Latihan3 > main.go > hitungMinMax
1 // Dharma Chandra Viriya
2 // SIIP-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 type arrBalita [100]float64
9
10 func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
11     *bMin = arrBerat[0]
12     *bMax = arrBerat[0]
13
14     for i := 1; i < n; i++ {
15         if arrBerat[i] < *bMin {
16             *bMin = arrBerat[i]
17         }
18         if arrBerat[i] > *bMax {
19             *bMax = arrBerat[i]
20         }
21     }
22 }
23
24 func rerata(arrBerat arrBalita, n int) float64 {
25     var total float64
26
27     for i := 0; i < n; i++ {
28         total += arrBerat[i]
29     }
30     return total / float64(n)
31 }
32
33 func main() {
34     n := 4
35     arrBerat := arrBalita{}
36     for i := 0; i < n; i++ {
37         fmt.Println("Masukan berat balita ke-", i+1)
38         var berat float64
39         fmt.Scan(&berat)
40         arrBerat[i] = berat
41     }
42     bMin := 0.0
43     bMax := 0.0
44     hitungMinMax(arrBerat, n, &bMin, &bMax)
45     rerata := rerata(arrBerat, n)
46     fmt.Printf("Berat balita minimum: %.2f kg\n", bMin)
47     fmt.Printf("Berat balita maksimum: %.2f kg\n", bMax)
48     fmt.Printf("Rerata berat balita: %.2f kg\n", rerata)
49 }
```

Terminal Output:

```
dharmachandravirya@Dharmas-MacBook-Pro go % go run Modul10/Latihan3/main.go
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
dharmachandravirya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini dibuat untuk mengolah data berat balita yang dicatat oleh petugas posyandu. Data berat balita disimpan dalam sebuah array bertipe `float64` dengan kapasitas maksimum 100 elemen. Pengguna terlebih dahulu memasukkan jumlah data berat balita yang akan dicatat, kemudian memasukkan berat masing-masing balita satu per satu. Setelah seluruh data berhasil disimpan, program akan menentukan berat balita minimum, berat balita maksimum, serta menghitung rerata berat seluruh balita. Hasil perhitungan tersebut kemudian ditampilkan dalam satuan kilogram dengan dua angka di belakang koma. Secara teknis, program menggunakan tipe bentukan `arrBalita` untuk merepresentasikan array data berat balita. Terdapat dua subprogram yang digunakan, yaitu prosedur `hitungMinMax` dan fungsi `rerata`. Prosedur `hitungMinMax` menerima array beserta jumlah data yang valid, kemudian mencari nilai minimum dan maksimum dengan menelusuri seluruh elemen array. Nilai minimum dan maksimum dikembalikan melalui parameter pointer `bMin` dan `bMax`, sehingga perubahan nilai dapat langsung tersimpan pada variabel yang berada di fungsi utama. Sementara itu, fungsi `rerata` bertugas menjumlahkan seluruh berat balita yang tersimpan dalam array, kemudian membaginya dengan jumlah data untuk memperoleh nilai rata-rata. Pada fungsi `main`, data dibaca dari masukan pengguna dan disimpan ke dalam array `data`, kemudian prosedur dan fungsi tersebut dipanggil untuk melakukan pengolahan data. Hasil akhirnya ditampilkan menggunakan `fmt.Printf` dengan format `%.2f` agar nilai berat ditampilkan dengan dua digit di belakang koma. Struktur program ini menunjukkan penerapan modularisasi melalui penggunaan prosedur dan fungsi sehingga kode menjadi lebih terorganisasi dan mudah dipelihara.

DAFTAR PUSTAKA